Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

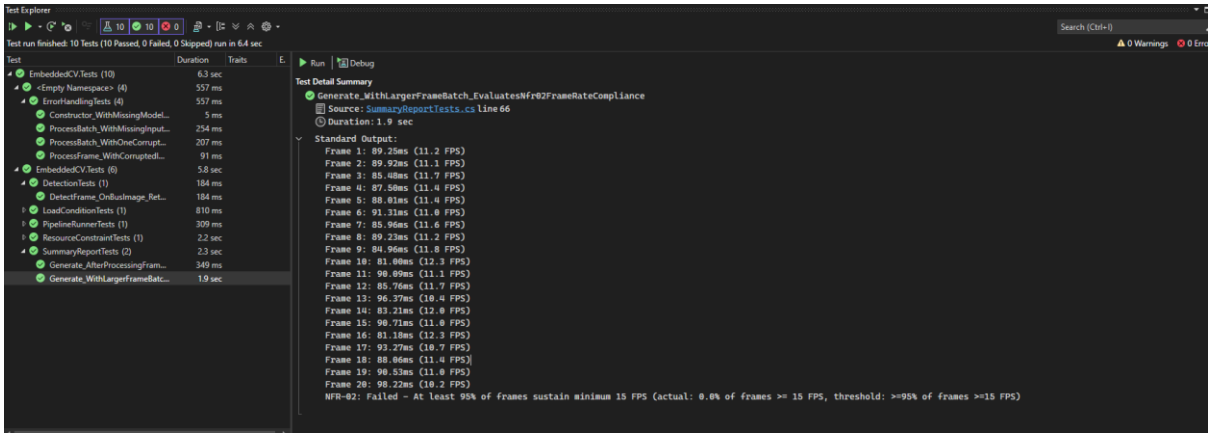
Task 1: Project Completion and Final Implementation

Requirements refinement (not scope change)

- NFR-09's cold-start threshold was changed from a placeholder of <2000ms to <500ms once real data (~75-100ms typical cold-start) showed the original threshold was too lenient.

```
//NFR-09: cold-start (model load + first inference) latency, tracked separately
if (coldStartLatencyMs.HasValue)
{
    report.NfrResults.Add(new NfrCheckResult
    {
        RequirementId = "NFR-09",
        Description = "Cold-start latency (model load + first inference)",
        Passed = coldStartLatencyMs.Value < 500.0, //not placeholder anymore, data has been analysed
        ActualValue = $"{coldStartLatencyMs.Value:F2}ms",
        Threshold = "<500ms"
    });
}
```

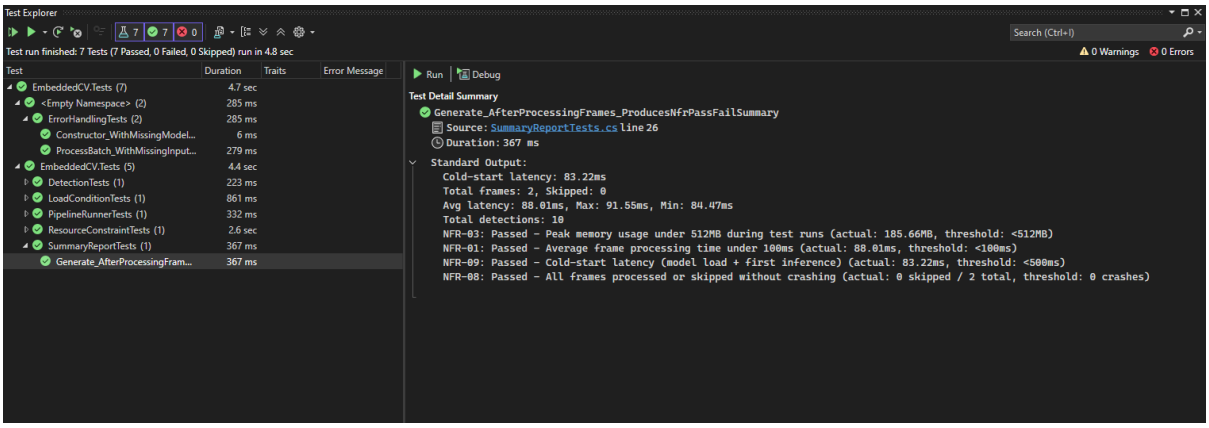
- NFR-02's threshold (>= 15 FPS for 95% of frames) was implemented and tested against real data, revealing the system always achieves ~10-12 FPS under baseline CPU-only inference which is an NFR failure, not an implementation gap/ This brings the question of whether 15 FPS is a realistic target for CPU-bound embedded-simulated inference, which we will revisit during quality metrics and release evaluation (Task 8) rather than treat it as a defect to silently fix by lowering the threshold.



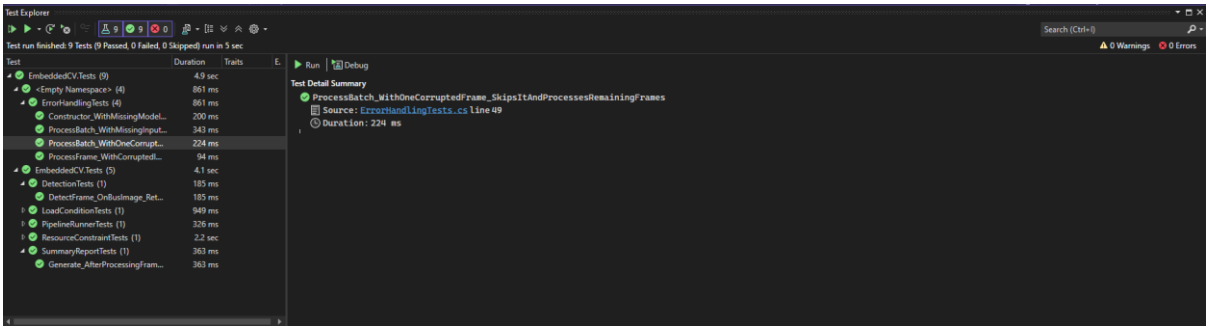
Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Implementation additions (filling gaps seen in Assessment 1 feedback)

- NFR-03 (memory usage) previously had a defined threshold with no implemented measurement; this has been implemented using process working set memory sampling and is now actively tested



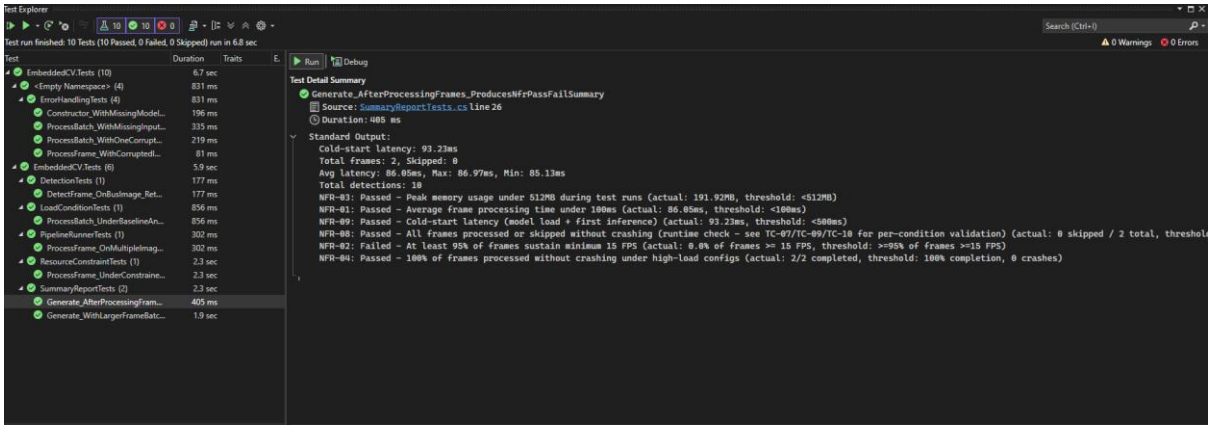
- NFR-08 (error handling) previously only verified that no unhandled exception occurred during a run; this has been changed with four dedicated test cases validating each specific error condition individually (missing model, corrupted frame, invalid input path), addressing the “false pass” risk seen in Assessment 1 feedback. Under ErrorHandlingTests are the four tests, covering Missing model (FR-09), Missing/invalid input path (FR-11), Single corrupted frame isolated (FR-10), and corrupted frame within a batch, confirming continuation (FR-10).



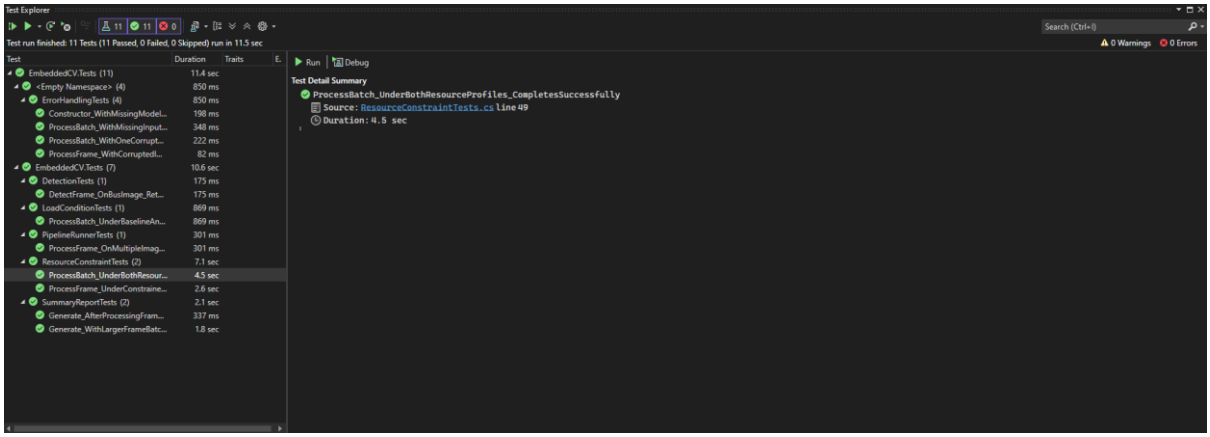
Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

- NFR-04, NFR-06, and NFR-07 previously untested, now have dedicated automated test coverage confirming reliability under load, consistency across repeated runs, and portability across hardware profiles.

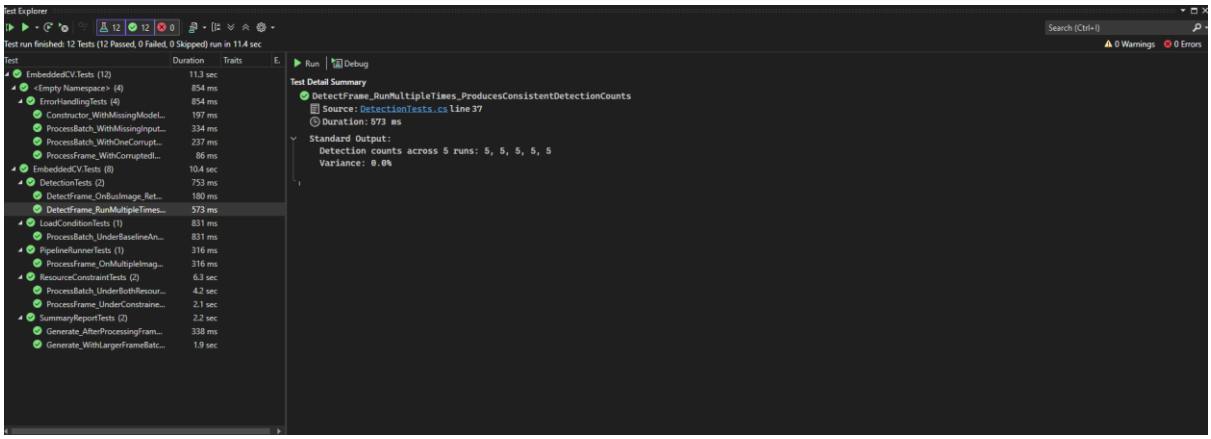
NFR-04



NFR-06



NFR-07

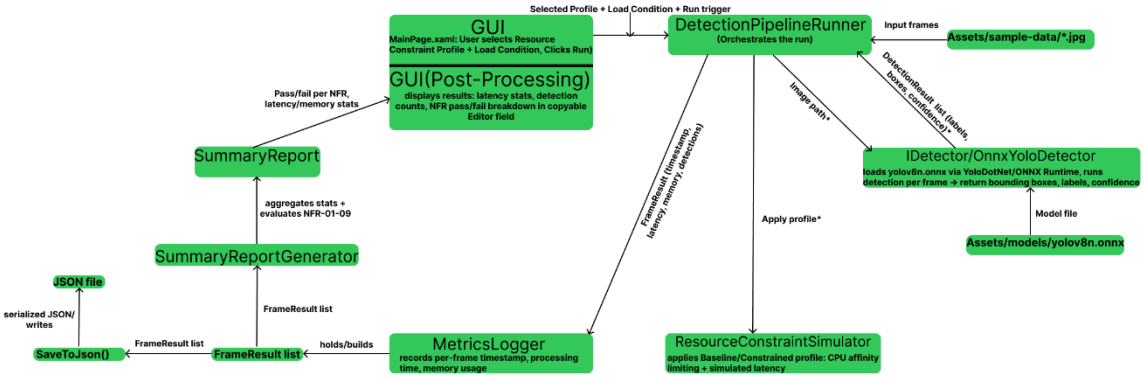


Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Architectural or technological changes

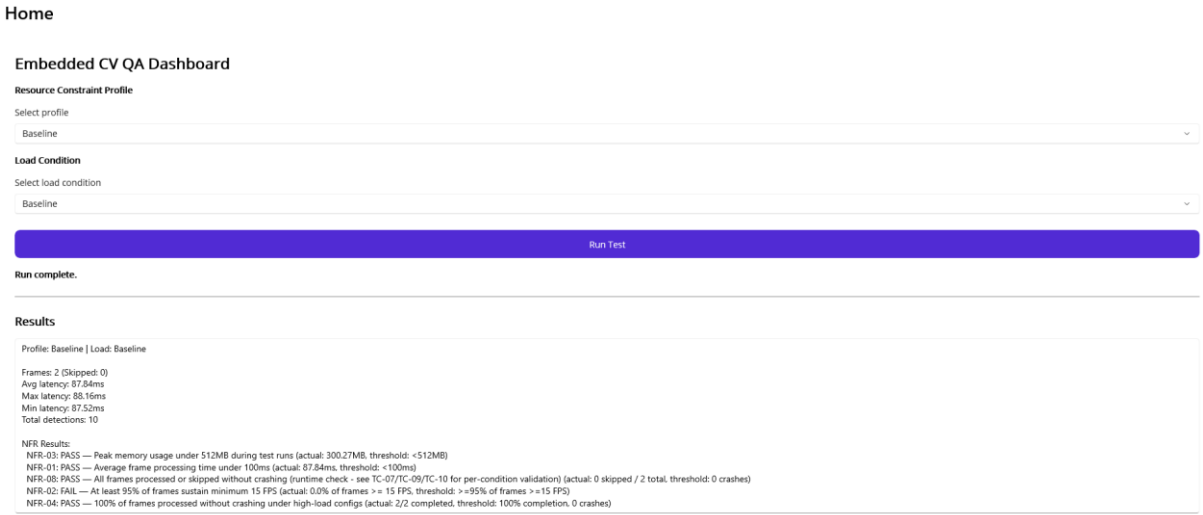
The core architecture (GUI → DetectionPipelineRunner → Detector/ResourceConstraintSimulator/MetricsLogging → SummaryReportGenerator) and the stack (C#/.NET 9, MAUI Windows, YoloDotNet/ONNX Runtime, MSTest) are unchanged from Assessment 1. Development since then is focused on finishing test coverage and measurement gaps seen in feedback, not restructuring the system.

Architecture/Data Flow diagram



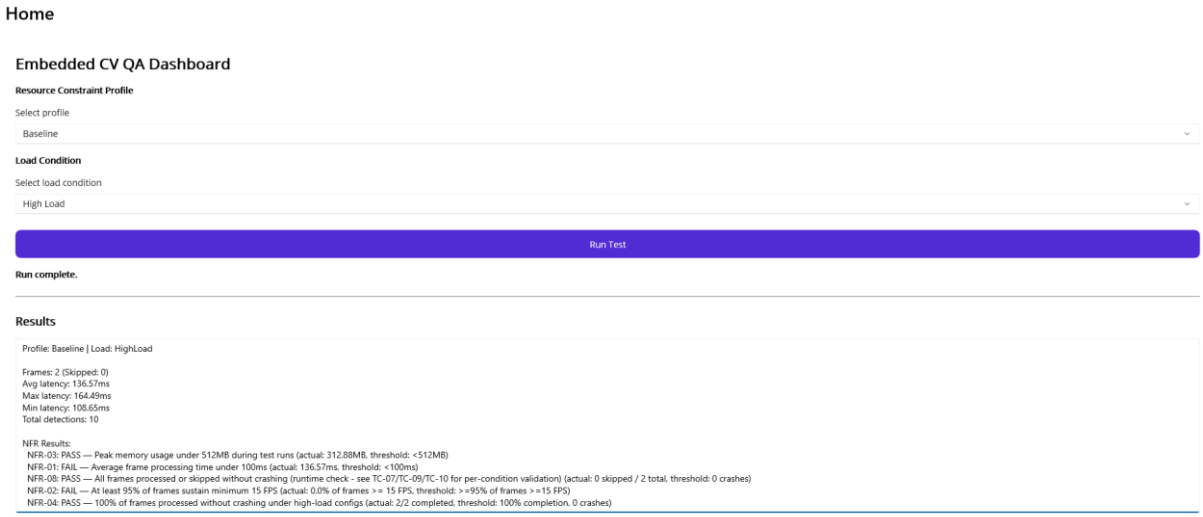
***: For the setup of those marked, Image path and DetectionResult list are per-frame, repeated calls But Apply profile is a one-time setup call (before the per-frame loop begins)**

GUI(Profile: Baseline, Load condition: Baseline)

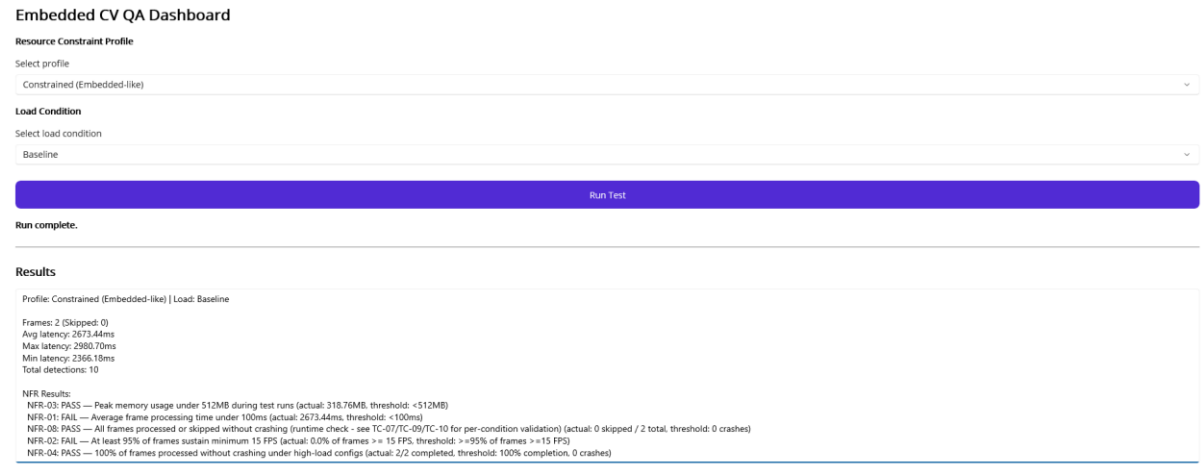


Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

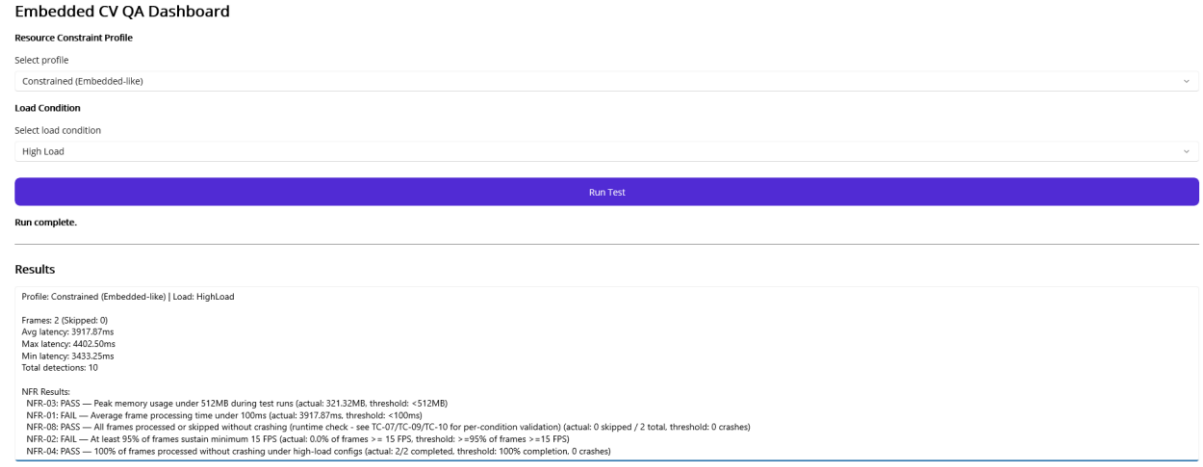
GUI(Profile: Baseline, Load condition: High load)



GUI(Profile: Constrained, Load condition: Baseline)



GUI(Profile: Constrained, Load condition: High load)



Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Deferred, not implemented

A feature, considered previously, which would allow users to upload their own dataset for testing (rather than the fixed sample set) was evaluated and deliberated on by Me (Yaacoub) and Matthew, we concluded that it doesn't map to any required Assessment 2 task and was judged not a good use of our time. This is documented as a future recommendation (Task 10) rather than abandoned.

NFR-05 (configurability) hasn't been automated, since it's fundamentally a GUI usability requirement rather than backend logic suited to unit testing. It'll be validated as a manual usability test case under Task 7 (Quality Testing), supported by GUI screenshots.

Task 2: Requirements Verification and Traceability

- Review and finalise the functional and non-functional requirements from Assessment 1.
- Update the Requirements Traceability Matrix.
- Map requirements to implemented features, test cases, test results, and defects where applicable.
- Identify any requirements that are partially implemented, unverified, or out of scope, and justify them.

Review and finalisation of requirements

The baseline is made up of the 11 functional requirements (FR-01 to FR-11) and the 9 non-functional requirements (NFR-01 to NFR-09) (including the updated and testable non-functional requirements, the error handling requirements (FR-09 to FR-11), and the Assessment 1 introduction of NFR-08 and NFR-09), which have remained constant since Assessment 1, with no additions or deletions made. Only the testing of these requirements has changed; NFR-03 is checked by a real memory measurement, NFR-08 by a different test for each error condition, and NFR-02, NFR-04, NFR-06, and NFR-07 are now tested with automated tests, whereas at mid-point, none of these four requirements had been tested.

The numbering of test cases here replaces the nine-test case numbering used in the traceability matrix for Assessment 1. The 15 FPS mark for NFR-02 stands, with the same failure in the current system; this is evaluated in Task 8 rather than

reducing the bar. Comparison of the baseline with the final system reveals three discrepancies between the code and the requirements.

FR-06 and NFR-07 mention CPU and memory usage limits, but the code only restricts the CPU to a single core under the constrained profile and adds a fixed delay per frame, with no memory limit; similarly, though FR-07 mentions detection accuracy, the summary report does not include an accuracy measure.

NFR-01 and NFR-02 are functionally overlapping, because at 15 FPS (a frame time of 66.7ms) any frame satisfying NFR-02 will also satisfy NFR-01's 100ms barrier. This is, however, not remedied (similarly to the above two points), to keep the requirements as originally written in Assessment 1.

Requirements Traceability Matrix

Test case IDs refer to the final test case table in Task 3. Defect IDs refer to the table that follows and are to be aligned with the defect register in Task 6.

ID	Requirement	Implemented in	Test case(s)	Result	Defect
FR-01	Load and run a pre-trained detection model on input data	OnnxYoloDetector (constructor)	TC-01	Pass	None
FR-02	Detect and classify objects in each frame	OnnxYoloDetector.DetectFrame	TC-01, TC-03	Pass (accuracy not verified)	None
FR-03	Output bounding boxes, labels and confidence scores	DetectionResult	TC-01, TC-03	Pass	None
FR-04	Process pre-recorded input as a simulated feed	DetectionPipelineRunner.ProcessBatch	TC-04, TC-11	Pass	None
FR-05	Log each frame's results and timestamp	MetricsLogger, FrameResult, SaveToJson	TC-04, TC-05	Pass	None
FR-06	Configurable simulated resource constraint before a run	ResourceConstraintProfile, ResourceConstraintSimulator, GUI profile picker	TC-06, TC-07	Partial (no memory cap)	None

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

ID	Requirement	Implemented in	Test case(s)	Result	Defect
FR-07	Summary report of accuracy and performance metrics	SummaryReportGenerator, GUI results panel	TC-08, TC-09, TC-10	Partial (no accuracy metric)	DEF-05
FR-08	Select between at least two load conditions	LoadCondition, ProcessBatch, GUI load picker	TC-11	Pass	None
FR-09	Stop and log an error if the model cannot load	OnnxYoloDetector (constructor)	TC-12	Pass	None
FR-10	Skip a corrupted frame, log it, continue	DetectionPipelineRunner.ProcessFrames	TC-14	Pass	None
FR-11	Reject a run before execution if input is missing or invalid	DetectionPipelineRunner.ValidateInputPaths	TC-13, TC-15	Pass	None
NFR-01	Frame processing time under 100ms	SummaryReportGenerator (NFR-01 check)	TC-06, TC-08	Pass for Baseline/Baseline (87.84ms); exceeded in the other three configurations	None
NFR-02	At least 95% of frames sustain 15 FPS	SummaryReportGenerator (NFR-02 check)	TC-09	Fail (0.0% of frames; about 11 FPS average)	DEF-01
NFR-03	Peak memory under 512MB	FrameResult.MemoryUsedBytes, SummaryReportGenerator (NFR-03 check)	TC-08, TC-10 (see note)	Pass (about 186 to 192MB in test runs, 300 to 322MB in GUI runs)	DEF-03 (closed)
NFR-04	100% of frames complete under high load	SummaryReportGenerator (NFR-04 check), Parallel.For in ProcessBatch	TC-11	Pass (2/2 completed in all four configurations)	None
NFR-05	Configure and launch a run in 3 steps or fewer, without editing source	MainPage.xaml (two pickers and a run button)	Manual GUI test (Task 7)	Not automated	None

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

ID	Requirement	Implemented in	Test case(s)	Result	Defect
NFR-06	Results within 2% variance across 5 repeated runs	DetectionTests variance check	TC-02	Pass (0% variance)	None
NFR-07	Full run completes under at least 2 simulated hardware profiles	ResourceConstraintSimulator, Baseline and Constrained profiles	TC-07	Pass	None
NFR-08	All defined error conditions handled without crashing	ProcessFrames try/catch, ValidateInputPaths, detector constructor, NFR-08 report check	TC-12, TC-13, TC-14, TC-15	Pass	DEF-04 (closed), DEF-05
NFR-09	Cold-start latency under 500ms	SummaryReportGenerator (NFR-09 check)	TC-10	Pass	None

Defect Mapping

ID	Defect or finding	Related requirement and test	Status
DEF-01	System sustains about 10 to 12 FPS on CPU-only YOLOv8n inference against a 15 FPS threshold.	NFR-02, TC-09	Open. Threshold review in Task 8.
DEF-02	Parallel test execution leaked the CPU affinity setting between tests, producing inconsistent latency results.	NFR-01, NFR-02	Closed. DoNotParallelize applied and affinity reset in test setup and cleanup.
DEF-03	NFR-03 had a defined threshold but no measurement in code.	NFR-03	Closed. Peak working set sampled per frame and checked in the report.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

ID	Defect or finding	Related requirement and test	Status
DEF-04	NFR-08 report check passed unconditionally if the run returned.	NFR-08, TC-12 to TC-15	Closed. Per-condition tests added. The report check remains a runtime check only.
DEF-05	The NFR-08 description string shown in the GUI and test output cites TC-07, TC-09 and TC-10, which do not match the final test case numbering (TC-12 to TC-15).	FR-07, NFR-08	Open. Low severity.
DEF-06	MSTestSettings.cs requests method-level parallel execution while AssemblyInfo.cs sets DoNotParallelize. The effective behaviour has not been confirmed.	NFR-01, NFR-02 (timing reliability)	Open. Low severity.

- **FR-06 (partially implemented).** The constrained profile forces the process onto a single CPU core, and has a further 50ms of emulated latency per frame. No memory cap has been implemented, so the constraint (FR-06/NFR-07) is a CPU-only constraint. Memory is monitored and verified against the 512MB limit (NFR-03) but not enforced. This was deemed suitable because the research objective is to investigate non-functional behaviour under emulation, and the two profiles are sufficient to demonstrate NFR-07.
- **FR-07 (partially implemented) and detection accuracy (unverified).** The report summarises the system: latency, RAM, number of detections and if NFRs pass or fail. It does not include a measure of detection accuracy, and no detection comparison to manually labelled samples has been performed, as required by the acceptance criteria of FR-02 and FR-03. Because TC-01 and TC-03 only check whether detections are produced, not whether they are the correct detections, an accuracy value cannot be derived from them. Accuracy of the YOLOv8n model is beyond

the scope of this project (verification of non-functional behaviour) and so has been marked as unverified.

- **NFR-02 (implemented, not passing).** TC-09 tests the requirement correctly and fails for all runs (0.0% of frames greater than or equal to 15 FPS). The test itself is passing in MSTest since the test only asserts the number of frames, so the failure appears in the report output and not as a red test in Test Explorer. Recorded as DEF-01 and taken to Task 8, where the realism of 15 FPS for CPU-only inference is considered.
- **NFR-01 (validated only for the baseline run).** The acceptance criterion is met in the baseline run (average latency 87.84ms). The same check exceeds 100ms in Baseline with High Load (136.57ms), Constrained with Baseline load (2673.44ms) and Constrained with High Load (3917.87ms). The two constrained values include the injected 50ms delay and single core limit so are to be expected. The wording is also inconsistent across the requirement, acceptance criterion and implementation: the rewritten requirement specifies each frame under 100ms, the acceptance criterion specifies 95% of frames and the implementation checks the average latency.
- **NFR-05 (verified manually).** Configurability is a GUI usability requirement and is not amenable to automated testing. Verified using a manual test case in Task 7 supported by the GUI screenshots for all four profile and load combinations.
- **NFR-06 (partially verified).** TC-02 calculates how much the count of detections changes in five runs of a single image. The reworded requirement is referring to class labels and bounding boxes and the test does not test the variance in bounding boxes, so consistency is only verified to the extent that it covers the need.
- **Sample data coverage (limitation).** All tests employ two base images, repeated in volume. Inclusion of an option for users to provide their own data set was initially slated for a future Task 10 recommendation, as this functionality does not appear to translate into a required Assessment 2 task.
- **Security, accessibility and compatibility (out of scope).** No requirements for these were defined in Assessment 1. The system is local-only, with no attack surface outside of the device, no interface relevant to accessibility except for one dashboard page, and one target platform (.NET MAUI on Windows).

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Task 3: Test Execution and Final Test Results

Execution Summary

Metric	Count
Planned	16
Executed	16
Passed	14
Failed	2 (TC-09, TC-16)
Blocked	0
Not Run	0

Explanation of Failed Test and Corrective Action

TC-09 (NFR-02, frame rate compliance) and TC-16 (NFR-01, NFR-02) are the only failed test cases. Both MSTest methods execute properly, and the failures reflect system behaviour rather than test or logic errors. In TC-09, the system sustains about 10-12 FPS on CPU-only YOLOv8n inference over a 20-frame sample, below the 15 FPS threshold. In TC-16, NFR-01 (100ms) is met only under Baseline/Baseline (85.1ms) and NFR-02 fails in every combination (best case 11.7 FPS). These are recorded as NFR non-conformances and not as code defects to fix, since they show the problem, this project set out to investigate: functional correctness does not guarantee real-time performance compliance. The proper action is at requirements level: whether 15 FPS is realistic for CPU-only inference, and how NFR-01 should apply under simulated constrained and high-load conditions, will be resolved in the quality metrics and release evaluation (Task 8). Neither result is a regression, as both have been consistent since they were first measured.

Status Convention

Status reflects whether the system met the test's expected result, NFR thresholds included. TC-09 and TC-16 are marked Fail because the system does not meet NFR-01 and NFR-02, even though their MSTest methods execute successfully and CI reports all 16 test methods passing.

Test Case Table

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

ID	Test Case	Preconditions	Steps	Input	Expected Result	Requirements	Type	Priority	Status
TC-01	Detector returns valid detections on known sample image	Model + sample image available	1. Load detector 2. Call DetectFrame(bus.jpg)	bus.jpg	Returns list of detections with labels, confidence, bounding box	FR-01-04	Unit/Integration	High	Pass
TC-02	Detection results consistent across repeated runs	Model loaded, warmed up	1. Run DetectFrame 5x on same image	bus.jpg x5	Detection count variance ≤2%	NFR-06	Non-functional	Medium	Pass (0% variance)
TC-03	Detector returns empty list on blank image	Model loaded	1. Generate blank image 2. Run DetectFrame	Blank white image	Returns empty detection list, no error	FR-02, FR-03	Boundary	Medium	Pass
TC-04	Full pipeline logs results and timing for multiple frames	Model + 2 sample images available	1. Process bus.jpg, frame 1 2. Process zidane.jpg, frame 2	bus.jpg, zidane.jpg	2 FrameResults logged with valid detections, timestamps, latency	FR-04, FR-05	Integration	High	Pass
TC-05	Logged results export to valid, parsable JSON	Frames processed and logged	1. Call SaveToJson() 2. Deserialize output	Logged FrameResult list	File exists, deserializes without error, matches frame count	FR-05	Integration	Medium	Pass
TC-06	Constrained resource profile produces higher latency than baseline	Model loaded	1. Run under Baseline profile 2. Run under Constrained profile	bus.jpg, both profiles	Constrained latency measurably higher	FR-06, NFR-01	Non-functional	High	Pass
TC-07	System completes a full run under both resource profiles	Model + sample images available	1. Run batch under Baseline 2. Run batch under Constrained	bus.jpg, zidane.jpg	Both runs complete, 0 frames skipped	FR-06, NFR-07	Non-functional	High	Pass
TC-08	Summary report evaluates latency (NFR-01) pass/fail against threshold	Frames processed	1. Generate summary report	Logged FrameResults	NFR-01 result reflects accurate pass/fail vs 100ms threshold	FR-07, NFR-01	Non-functional	High	Pass
TC-09	Frame rate compliance (NFR-02) evaluated across larger batch	Model warmed up	1. Process 20-frame batch 2. Generate report	20 images (repeated samples)	NFR-02 result reflects ≥95% of frames sustain ≥15 FPS	FR-07, NFR-02	Non-functional	Medium	Fail (0% compliance, system achieves ~11 FPS avg)
TC-10	Cold-start latency measured separately from steady-state	Fresh detector instance	1. Measure first DetectFrame call 2. Process further frames	bus.jpg	Cold-start and steady-state latency reported separately	FR-07, NFR-09	Non-functional	Medium	Pass
TC-11	Batch processing completes under Baseline and HighLoad conditions	Model + sample images available	1. Run batch, Baseline load 2. Run batch, HighLoad	4 images	Both complete all frames, 0 skipped	FR-08	Functional	High	Pass
TC-12	Detector constructor throws clear error on missing model	No model file at given path	1. Construct OnnxYoloDetector with invalid path	Invalid model path	FileNotFoundException thrown with clear message	FR-09	Negative	High	Pass
TC-13	Batch run rejected before execution on missing input file	Model available	1. Call ProcessBatch with non-existent image path	Invalid image path	FileNotFoundException thrown, no frames processed	FR-11	Negative	High	Pass
TC-14	Corrupted frame is skipped and logged, run continues	Model available, 1 corrupted + 1 valid file	1. Process corrupted file alone 2. Process batch with corrupted + valid file	Corrupted file, bus.jpg	Corrupted frame marked skipped with reason; valid frame still processed	FR-10	Negative	High	Pass
TC-15	Empty batch request is rejected before execution	Model available	1. Call ProcessBatch with empty list	Empty list	ArgumentException thrown	FR-11	Boundary/Negative	Medium	Pass
TC-16	Performance across all resource-profile and load-condition combinations	Model and both sample images available; CPU affinity reset at start; a warm-up frame run under each profile	1. For each of Baseline/Constrained x Baseline/HighLoad, apply the profile 2. Run a warm-up frame, then process a 10-frame batch under the load condition 3. Generate the summary report and record latency, throughput and NFR results	10 frames per combination, alternating bus.jpg and zidane.jpg (40 recorded frames)	All 4 combinations complete 10/10 frames with 0 skipped; NFR-01 average latency <100ms; NFR-02 ≥95% of frames at ≥15 FPS; NFR-03 peak memory <512MB	NFR-01, NFR-02, NFR-03, NFR-04, NFR-07	Non-functional (performance/reliability)	High	Fail: completion, NFR-03, NFR-04 and NFR-07 met; NFR-01 failed in 3 of 4 combinations (only Baseline/Baseline passed, 85.1ms) and NFR-02 failed in all 4 (best case 11.7 FPS)

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Regression Testing

The full MSTest suite is re-run after every code change via Visual Studio's Test Explorer, following the regression practice after the Assessment 1 concurrency bug (where parallel test execution caused leakage between tests). [assembly: DoNotParallelize] stays to guarantee consistent, repeatable results across runs, and all 16 test cases are re-verified as part of this regression process before each commit.

Remaining Quality Risks

- The NFR-02 failures (TC-09, TC-16) and the NFR-01 failures under Constrained and HighLoad conditions (TC-16) remain open and feed the Task 8 release decision.
- NFR-05 (configurability) is still validated manually rather than through automated testing (See Task 7), which holds a small risk of being under-documented compared to the automated suite.
- Test coverage relies on a small, fixed sample image set (2 base images, reused/repeated for volume), which lowers confidence in detection-accuracy claims across real-world conditions.

Task 4: Test Automation and Quality Engineering

- Identify suitable test cases for automation and justify the selection.
- Implement automated tests using an appropriate tool or framework where suitable.
- Report automated test execution results using logs, screenshots, reports, or repository evidence.
- Discuss the benefits, limitations, and maintainability of the automated test suite.
- Where automation is not appropriate, justify the use of manual or exploratory testing.

Selection of test cases for automation

Test cases were selected for automation when they had a known input, a measurable expected result, and a cost to repeat by hand that was higher than the cost of writing the test. By these criteria all 16 test cases in Task 3 (TC-01 to TC-16) were automated. The tests fall into five groups.

Group	Test cases	Justification

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Detection Correctness	TC-01, TC-03	A fixed sample image and a blank image give known expected outputs, so the result is the same on every run.
Pipeline and logging	TC-04, TC-05	These check the full path from frame to logged result and JSON export, which has to be re-verified after every code change.
Resource profiles and load conditions	TC-06, TC-07, TC-11, TC-16	The requirements need runs under two profiles and two load conditions. Doing this by hand for every change would be slow and inconsistent.
NFR measurement	TC-02, TC-08, TC-09, TC-10	Threshold for variance, latency, frame rate and cold start are calculated in code, so a script is more accurate than a person reading a screen.
Error Handling	TC-12 to TC-15	A missing model, a corrupted frame, a missing input file and an empty batch are hard to trigger consistently by hand and cheap to automate.

Implementation

Tests written in MSTest (.NET 9) are discovered and run through the Visual Studio Test Explorer. These consist of a set of 16 tests split across 7 classes: DetectionTests (3), ErrorHandlingTests (5), LoadConditionTests (1), PipelineRunnerTests (2), ResourceConstraintTests (2), SummaryReportTests (2) and PerformanceMatrixTests (1). These run the actual YOLOv8n model on images and do not use mocks, ensuring that the same code path is executed as in the GUI. In addition, the thresholds are evaluated by the summary report generator, so the GUI and the automated test suite use the same set of non-functional requirement validation checks. The tests reset CPU affinity in TestInitialize and TestCleanup, and use the DoNotParallelize attribute (added as the fix for DEF-02 in Task 6 to ensure tests do not interfere with each other). Finally, the test suite is run on every push to main and every pull request into main via the GitHub Actions workflow discussed in Task 5, with .trx results uploaded as artifacts.

Execution results

The execution and completion of all 16 test methods occur normally, with passing results in MSTest and inside the CI environment. Task 3 records that 14 test cases passed, while 2 (TC-09 and TC-16) failed. This is because the status in Task 3 reflects whether the NFR thresholds were satisfied. The outputs of TC-09 and TC-16 report the thresholds as not met, however this is not asserted; the reasoning for this is that absolute timings are machine specific and hence

unreliable for CI, so the failure appears in the report output rather than as red tests inside Test Explorer.

Test Explorer results for NFR-02, NFR-03, NFR-04, NFR-06, NFR-07 and NFR-08 are shown in Task 1, and the full set of screenshots is in the root of the repository, with file names starting NFR-02, NFR-03 and so on. CI evidence is shown in Task 5.

Benefits

The automated suite can re-run the complete pipeline within seconds or minutes of each and every change to it. In addition, the whole suite is run by CI after every push to the main branch and every pull request into the main branch. It evaluates the NFR thresholds consistently on every run and found real issues such as the 15 FPS gap (DEF-01) and the affinity leakage issue during execution of parallel tests (DEF-02), and it guards against a false NFR-08 pass (DEF-04).

Limitations

Timing Sensitivity

The system upon which a process executes impacts its timing, and as such a host's load will affect the resulting latency. For this reason, if tests are performed close to a certain threshold they may either pass or fail depending on what other tasks that host machine is executing concurrently. Timing thresholds are therefore reported, not asserted.

Failures can be hidden in the test result

Some failures are not visible in the test result (tests are seen as successful in Test Explorer even though the NFR is not met): TC-09 and TC-16, whose thresholds are not asserted.

Input set

The tests only use two sample images, meaning detection accuracy is not verified, and bounding box variance is not being tested either.

Coverage/Mocking

Finally, code coverage was not measured, and coverage thresholds were not defined (see Task 5). Furthermore, there is no use of mocks via IDetector, meaning pipeline logic cannot be tested without the presence of the model file.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Maintainability

The tests are structured around the primary areas of interest with each class relating to the appropriate area of the system. Test names are descriptive of the behaviour which is verified. Threshold values for NFRs are centralized in the summary report generator rather than embedded in each test so changing a threshold shouldn't require code changes to the tests. Nonetheless, maintainability is reduced by two things:
The affinity reset function is currently copied into 5 of the 7 test classes and this should be moved to a shared base class. Also still open are DEF-05 (stale test reference mentioned in the description of NFR-08) and DEF-06 (conflicting parallel setting).

Where automation is not appropriate

- The usability requirement NFR-05 (3 steps to configure and execute) applies to the user experience when using the GUI. Therefore, this has been verified manually via screenshots and the use of independent testers (Task 7).
- Performing exploratory testing upon the GUI has raised issues that are unlikely to have been caught by the scripted testing. Notably: a run with no selection that defaults to Baseline/Baseline without a message (DEF-07) and option labels which do not fully describe what they do (DEF-08).
- Automating the detection accuracy check was not done because it would require the construction of a labelled data set, which was outside the remit of this project (Task 2).

Task 5: Continuous Integration and DevOps QA

Development and Integration Workflow

Development follows a feature-branch workflow: each team member works on a specific branch for their assigned tasks, committing regularly (minimum 2 commits/week) before merging into main via GitHub at the end. This branch-per-task structure, in place since Assessment 1, keeps individual contributions traceable and reviewable, which supports the GitHub-evidence requirement for the contribution statement (Task 11). CI was implemented during Assessment 2 and runs when changes are pushed to main or a pull request into main is opened, so work on feature branches is checked at merge rather than on every individual commit.

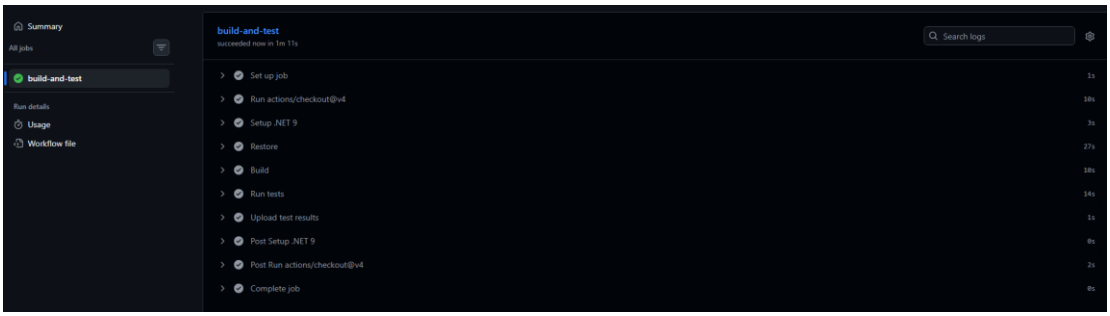
Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

CI Workflow Implementation

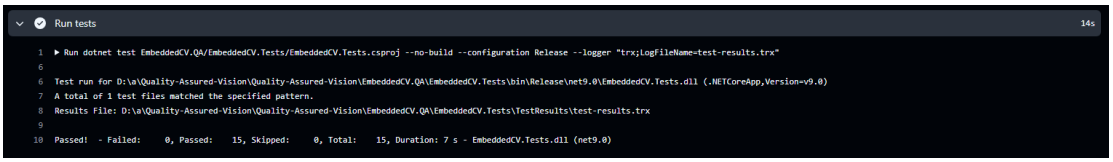
A GitHub Actions workflow was made to automatically build and test the EmbeddedCV.Tests project (and EmbeddedCV.Core via references) on a Windows runner on every push and pull request to main and can also be triggered manually. The MAUI GUI project is excluded because it needs the Windows desktop workload and the tests don't depend on it. This returns automated feedback on changes that break the build or test suite.

(See .github/workflows/ci.yml for the file)

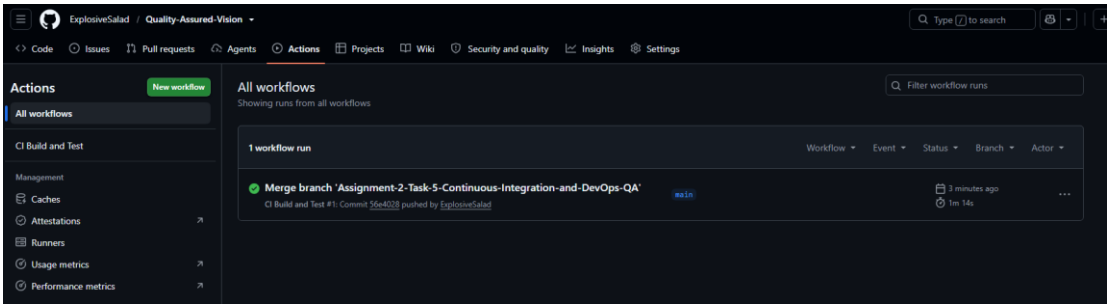
CI Build test:



Run Tests Step:



Proof of the CI Action in Github:



Quality Gates Defined

- **Successful build:** The workflow fails if the restore or build step fails, flagging broken code.
- **All tests passing:** The workflow fails if any of the 16 tests fail. NFR threshold results are reported in the test output but don't gate the build. Many depend on hardware speed and would make CI unreliable on GitHub's runners.

- **Test result retention:** .trx results are uploaded as artifacts every run, giving a reviewable record of which tests passed/failed on each commit.

Linting, coverage thresholds and enforced code review were not implemented in this phase.

Smoke/Sanity Testing

The CI run acts as a sanity check. A successful build confirms the solution compiles with all references resolving, and the test run confirms the core pipeline (detection, logging, reporting, error handling) still works after each change.

Task 6: Defect Management and Root Cause Analysis

- Maintain a final defect register covering defects found during development and testing.
- Define severity, priority, and defect status categories.
- Describe the defect lifecycle used in the project.
- Analyse at *least three (3)* significant defects using root cause analysis.
- Propose corrective actions and defect-prevention measures.
- Explain how defect analysis improved product or process quality.

Severity, priority and status change

Category	Value	Meaning
Severity	High	Produces invalid test results or breaks a core function.
	Medium	A requirement isn't met or a check is unreliable, however the system still runs correctly.
	Low	Cosmetic, wording or usability issue that doesn't affect the results
Priority	High	Needs to be fixed before next test run
	Medium	Needs to be Fixed or resolved before final release evaluation.
	Low	Fixed if time allows it to be, otherwise recorded as a limitation
Status	Open	Found and recorded but not yet resolved.
	Under review	Needs a decision, not a code change.
	Closed	Fixed and verified by re-running the test

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Defect lifecycle

The defects discovered, either by automated running of the MSTest tests from Visual Studio's Test Explorer and their summary report output or by our manual evaluation of the GUI, were entered into the defect register. Each defect was cross-referenced to its requirement and test case and given a severity and priority rating. Code defects were fixed and then re-verified by running the complete MSTest suite according to the regression testing discussed in task 3. Requirement-based defects (DEF-01), however, were not fixed and were escalated as review decisions, since they were not code errors. Closed defects were also checked against the traceability matrix. DEF-03, DEF-04, and DEF-06 were found by reviewing the code and requirements, not by running a test or using the GUI.

Final Defect Register

ID	Defect	Related requirement and test	Severity	Priority	Status
DEF-01	System sustains about 10 to 12FPS on CPU-only YOLOv8n interface against a 15FPS threshold.	NFR-02, TC-09	Medium	Medium	Under review (Task 8)
DEF-02	Parallel test execution leaked the CPU affinity setting between tests, producing inconsistent latency results.	NFR-01, NFR-02	High	High	Closed
DEF-03	NFR-03 had defined threshold but no measurement in code	NFR-03	Medium	High	Closed
DEF-04	NFR-08 report check passed unconditionally if the run returned	NFR-08, TC-12 to TC-15	High	Medium	Closed
DEF-05	NFR-08 description string cites TC-07, TC-09, and TC-0, which do no match final test case numbering (TC-12 to TC-15.)	FR-07, NFR-08	Low	Medium	Open

DEF-06	MSTestSettings.cs requests method-level parallel execution while AssemblyInfo.cs sets DoNotParallelize. The effective behaviour has not been confirmed	NFR-01, NFR-02	Medium	Low	Open
DEF-07	Profile and load pickers start blank, and a run with no selection uses Baseline/Baseline without telling the user	NFR-05	Low	Low	Open
DEF-08	Option labels do not explain what each profile or load condition does	NFR-05	Low	Low	Open

Root Cause analysis

DEF-02: CPU affinity leakage between parallel tests**.** The constrained profile constrains the process to a single core by changing process affinity. The change is then made to the process, not to a specific test. MSTest was running the tests in parallel, and so a constrained test would change the core count for a different running test. Latency results therefore varied depending on what other tests were running in parallel. The root cause was shared process-level state and parallel test execution, which didn't isolate one test from the next. The fix was to add the DoNotParallelize attribute and reset the affinity when the test is set up and cleaned up. As a preventative measure, the entire suite is re-run after each change (Task 3). DEF-06 records that a conflicting parallel setting remains in the test project and its effective behaviour needs confirmation.

DEF-04: NFR-08 check always succeeded**.** The requirement states that all defined error conditions are handled without crashing. The report check only verified that the run finished, so it would pass even if an error handler was broken. The root cause was that this broad requirement was implemented as a runtime flag without individual tests for each error condition. The fix was to add tests for each condition: missing model, corrupted frame, missing input file, and empty batch (TC-12 to TC-15). The report check remains a runtime-only check. As a preventative measure, each NFR is mapped to a test that can fail, as shown in the Task 2 traceability matrix.

DEF-01: FPS too low (below NFR-02). The 15 FPS threshold was specified in the Assessment 1 requirements before the system was measured. The test results show that CPU-only YOLOv8n inference reaches only about 10 to 12 FPS, hence

the requirement fails even though the code is correct. The root cause is, therefore, that the requirement was established without a measured baseline rather than incorrect code. The proposed fix is a requirement-level review in Task 8; the threshold should not be lowered silently to make the test pass.
As a preventative measure, performance requirement thresholds should be set after establishing a baseline.

How defect analysis improved quality

The defects have had a major effect on the product and testing. Evidence from DEF-03 and DEF-04 indicates that the requirement NFR-03 has now been tested correctly and that NFR-08 is tested under conditions that can fail. DEF-02 has also introduced sequential testing, and a regression run after each change has made the timing results more consistent. DEF-01 shows that a failing requirement might not be due to a code problem but due to a requirement problem, and this is carried into the release evaluation in Task 8. The usability defects recorded as DEF-07 and DEF-08 have led to the recommended improvements outlined in Task 7.

Task 7: Quality Testing

Area 1: Usability

Risk-based justification

The GUI is the only way a tester configures and runs a scenario, so a confusing interface leads to improper runs and misread results. That would undermine the projects' true purpose, which is trustworthy NFR measurements. Usability also covers NFR-05, the only requirement that can't be automated, so it needs manual evidence to be verified.

Approach

Manual, scenario-based testing of MainPage against NFR-05. Each test was done on the finished GUI, with screenshots as evidence where applicable.

Test cases

ID	Test	Preconditions	Steps	Expected	Priority	Status

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

UT-01	Configure and run in ≤3 steps, no code edits (NFR-05)	App launched, MainPage idle	1. Select resource profile 2. Select load condition 3. Click Run Test	Run completes; results header shows the selected profile and load	High	Pass

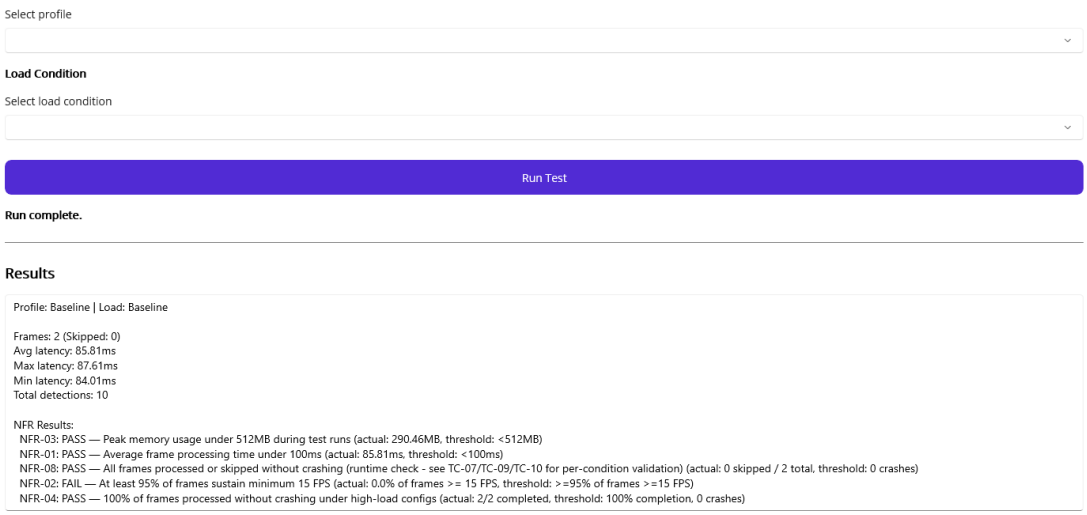


Configured and run in 3 steps (profile, load condition, Run) with no code edits. Meets NFR-05.

ID	Test	Preconditions	Steps	Expected	Priority	Status

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

UT-02	Run with no selections made	App freshly launched	Click Run Test without touching either dropdown	Run starts with sensible defaults, or the user is told what's missing	Medium	Pass with observation

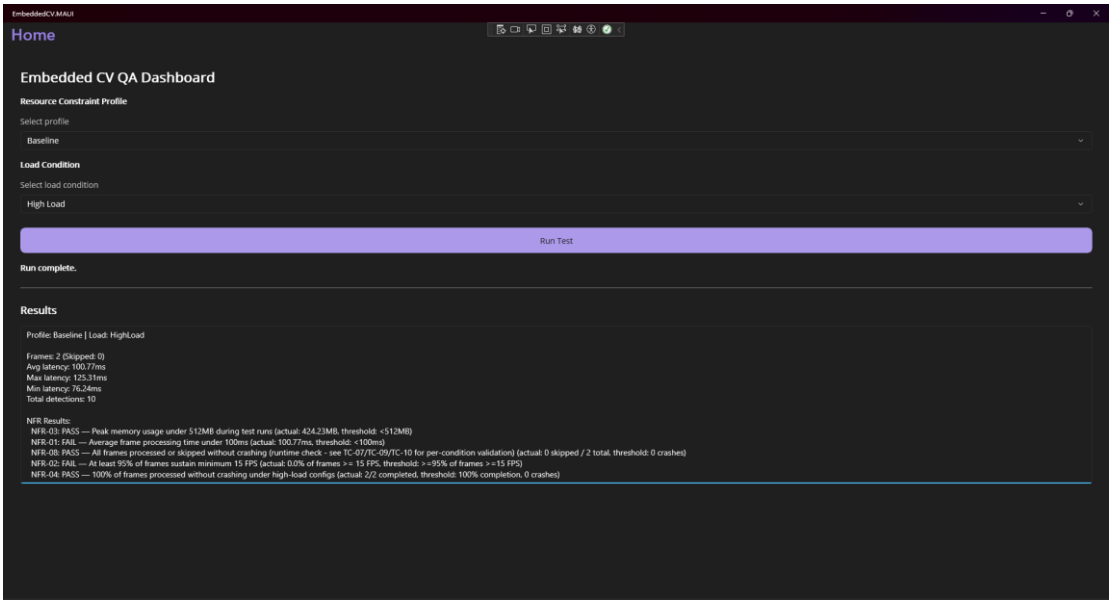


Running with no selections defaulted to Baseline/Baseline. The results header showed the profile, but nothing told the user a default was applied.

ID	Test	Preconditions	Steps	Expected	Priority	Status

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

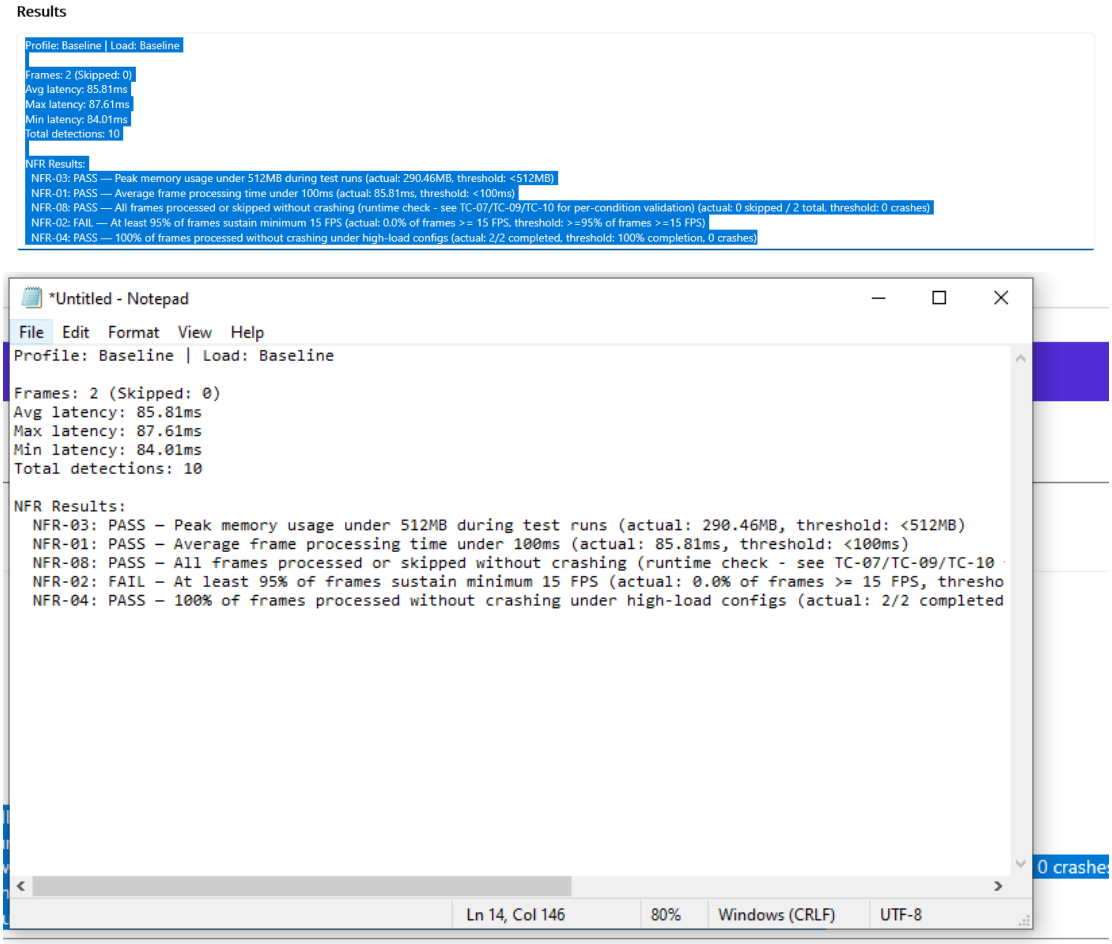
UT-03	First-time user completes UT-01 unaided	Tester who didn't build the GUI	Give the tester only the instruction "run a constrained , high-load test"	Completes in ≤3 steps without help	High	Pass (limited evidence)



Matthew, who didn't build the GUI, found it easy to understand, and completed it in three steps.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

ID	Test	Preconditions	Steps	Expected	Priority	Status
UT-04	Results readable and copyable	A completed run	Select the results text and copy it into a text editor	Full summary copies intact	Low	Pass



The full results summary was readable and copied into a text editor.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

ID	Test	Preconditions	Steps	Expected	Priority	Status
UT-05	Unprompted use by an independent tester	App launched, MainPage idle, tester unfamiliar with the project, given no instructions or suggested settings, and unaware of being timed	Tester explores the GUI and runs a test of their own choosing	Tester configures and starts a run unaided in ≤3 steps	Medium	Pass: completed in 3 steps, 7.11s. Tester chose Constrained + High Load; said the option labels didn't make clear what each executes

Findings

NFR-05 is met. The GUI lets a user configure and launch a run in three steps without touching source code, and two other people, Matthew and Devon (an independent tester) found it easy to follow. Two low-severity observations come out of testing. First (UT-02), the pickers start blank and a run with no selection uses Baseline/Baseline, so a user could think they ran a constrained test when they didn't. Second (UT-05), the option labels don't explain what each profile or load condition does. Both are recorded in the Task 6 defect register as low-severity, low-priority usability defects. **Matthew, Log this in Task 6 defect register as a low-severity, low-priority usability defect.**

Limitations

- Only two people other than the developer tested the GUI. One is a teammate who knows the project and the other a friend with no prior knowledge, so this is supporting evidence and not a usability study. UT-03 has no recorded timing.
- No formal usability scale (such as SUS, System Usability Scale) and no accessibility testing.
- The GUI only runs the fixed sample images, so real-world workflows aren't covered.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Recommended Improvements

- Preselect Baseline in both pickers
- Use descriptive labels (e.g. “Constrained: 1 core, +50ms simulated latency”)
- Show the active configuration next to the Run button
- Repeat testing with several unfamiliar testers using timed tasks.

Area 2: Performance

Risk-based justification

The project’s central claim is that a system can be functionally correct and still fail real-time requirements. NFR-01, NFR-02, and NFR-03 (latency, frame rate, memory) are the requirements most likely to fail in deployment, so they hold the highest risk and are tested first.

Approach

TC-16 runs a 10-frame batch under every combination of resource profile (Baseline, Constrained) and load condition (Baseline, HighLoad), after a warm-up frame in each cell, and records average per-frame latency, wall-clock throughput, and the NFR-01/02/03 results. The test asserts only that every frame completes. Thresholds are reported but not asserted, because absolute timings vary based on the host machine and would make CI unreliable.

Profile	Load	Avg latency (per frame)	Throughput (frames/wall time)	NFR-01	NFR-02	NFR-03 (peak)
Baseline	Baseline	85.1ms	11.7 FPS	Pass	Fail	Pass (180MB)
Baseline	HighLoad	407.8ms	12.5 FPS	Fail	Fail	Pass (214MB)
Constrained	Baseline	1892.8ms	0.5 FPS	Fail	Fail	Pass (211MB)
Constrained	HighLoad	6004.5ms	0.6 FPS	Fail	Fail	Pass (234MB)

Findings

Only the unconstrained, baseline-load case meets NFR-01, and no case meets NFR-02, since even the best case sustains about 11.7 FPS against the 15 FPS target. Memory is not a concern. Constrained runs are 22x slower per frame; the cause (single-core pinning, and possible thread contention inside the inference

runtime) was not isolated. Under HighLoad, per-frame latency increases about 4.8x while total throughput stays roughly the same, so the NFR-01 failure under load is consistent with queuing rather than slower processing, even though this wasn't isolated. NFR-02 is calculated from per-frame latency, so it understates throughput under concurrency. The wall-clock figure is also below 15 FPS, so the result is unchanged. All four cells completed every frame with none skipped, supporting NFR-04 and NFR-07.

Limitations

Each cell was run once, so run-to-run variance is unmeasured. Results come from one machine (Ryzen 7 2700X, 8 core processor, 32GB DDR4 RAM) and two sample images. Memory is the process-wide working set, which isn't reset between cells, so the values aren't independent. Constraints are simulated and not measured on embedded hardware. CI runners will give different figures, so the report uses local results.

Recommended Improvements

Repeat each cell several times and report mean and spread, treat wall-clock throughput as an explicit metric alongside per-frame latency, and review the 15 FPS and 100ms thresholds against what CPU-only inference can achieve.

Task 8: Quality Metrics and Final Release Decision

- Present relevant quality metrics, such as test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, or commit activity.
- Evaluate the completed system against requirements, acceptance criteria, and quality attributes.
- Summarise testing results, defect status, metrics, limitations, and residual risks.
- State whether the system is ready for release, conditionally ready, or not ready for release.
- Justify the release decision using evidence.

Task 9: AI-Assisted Development and Critical Reflection

- Explain how GitHub Copilot was used for implementation, testing, debugging, refactoring, or QA activities.
- Provide representative evidence of AI-assisted development.

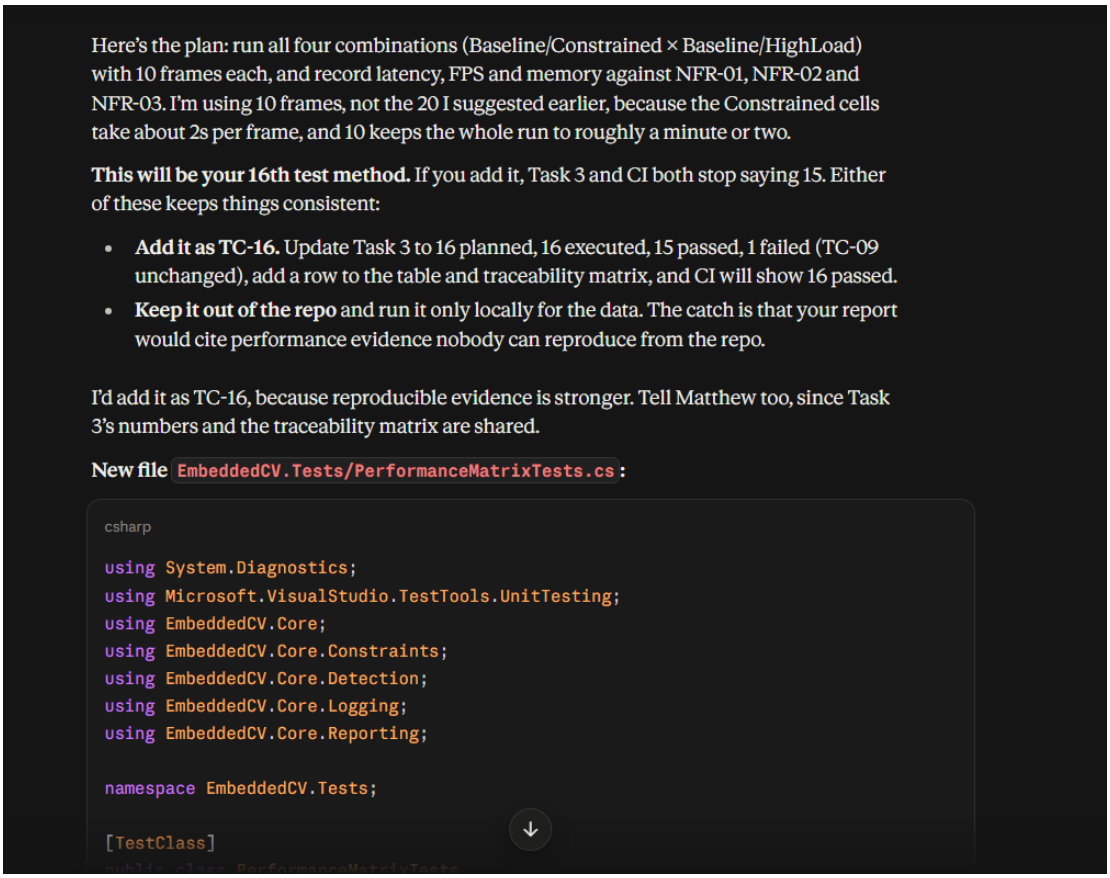
Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

- Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected.
- Identify any quality risks found in AI-assisted outputs, such as incorrect logic, insecure code, poor maintainability, or unsuitable assumptions.
- Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice.

How AI was used

Two tools were used. GitHub Copilot in Visual Studio gave us inline and chat suggestions whilst coding, and Claude was used as an assistant for scaffolding (detector, constraint simulator, metrics logger, report generator, GUI), drafting MSTest cases, diagnosing errors and helped plan the CI workflow.

Representative evidence



Here's the plan: run all four combinations (Baseline/Constrained × Baseline/HighLoad) with 10 frames each, and record latency, FPS and memory against NFR-01, NFR-02 and NFR-03. I'm using 10 frames, not the 20 I suggested earlier, because the Constrained cells take about 2s per frame, and 10 keeps the whole run to roughly a minute or two.

This will be your 16th test method. If you add it, Task 3 and CI both stop saying 15. Either of these keeps things consistent:

- **Add it as TC-16.** Update Task 3 to 16 planned, 16 executed, 15 passed, 1 failed (TC-09 unchanged), add a row to the table and traceability matrix, and CI will show 16 passed.
- **Keep it out of the repo** and run it only locally for the data. The catch is that your report would cite performance evidence nobody can reproduce from the repo.

I'd add it as TC-16, because reproducible evidence is stronger. Tell Matthew too, since Task 3's numbers and the traceability matrix are shared.

New file `EmbeddedCV.Tests/PerformanceMatrixTests.cs`**:**

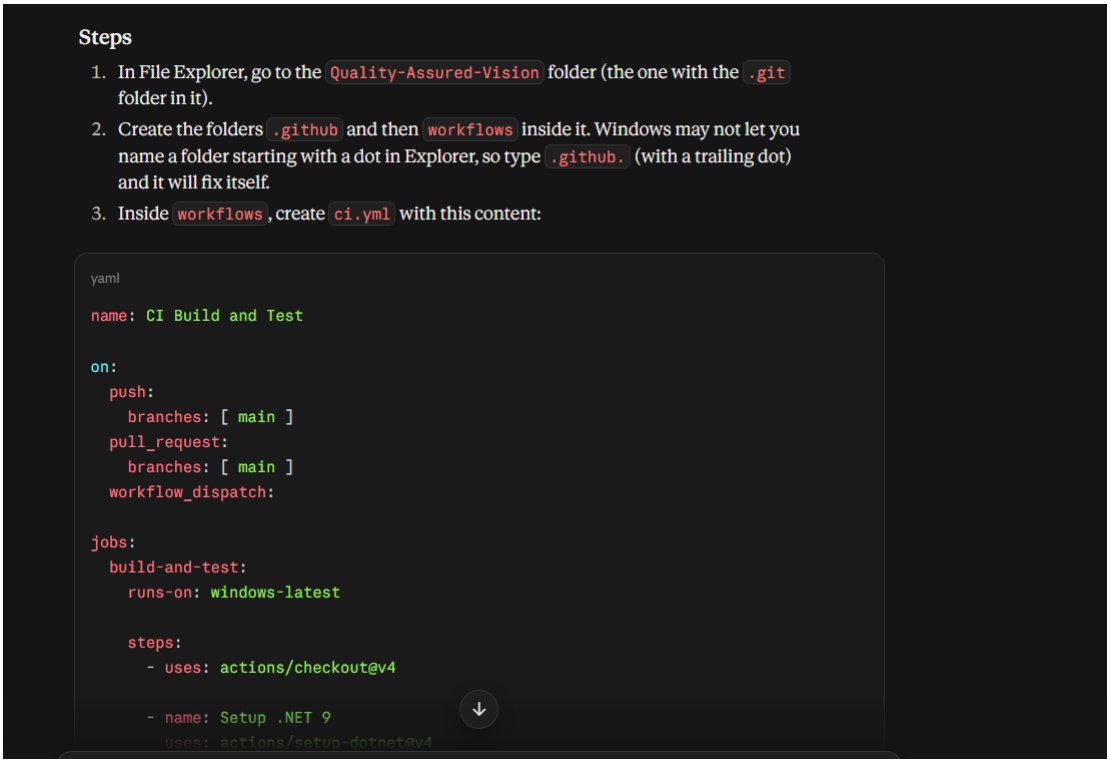
```csharp
using System.Diagnostics;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using EmbeddedCV.Core;
using EmbeddedCV.Core.Constraints;
using EmbeddedCV.Core.Detection;
using EmbeddedCV.Core.Logging;
using EmbeddedCV.Core.Reporting;

namespace EmbeddedCV.Tests;

[TestClass]
```

I keep claude in the loop with everything we've done thus far (Project with matthew, what we both write in the report, how its shaping out), this is giving us a new test case and making sure that I update my Tasks and keeping up to date, not directly writing the report.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git



Steps

1. In File Explorer, go to the `Quality-Assured-Vision` folder (the one with the `.git` folder in it).
2. Create the folders `.github` and then `workflows` inside it. Windows may not let you name a folder starting with a dot in Explorer, so type `.github.` (with a trailing dot) and it will fix itself.
3. Inside `workflows`, create `ci.yml` with this content:

```yaml
name: CI Build and Test

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  build-and-test:
    runs-on: windows-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET 9
        uses: actions/setup-dotnet@v4
```

This shows claude helping me build the CI workflow because I lack the knowledge in CI workflows.

Sadly, I could not get the Copilot images because the chatlogs are gone.

Review, modification and testing

Suggestions weren't taken at face value. Library APIs were checked against decompiled source, code was built and run, and each module got MSTest coverage. A latency anomaly (890-3001ms against ~71-85ms in clean runs) was isolated and reproduced. The first fix (resetting affinity before and after tests) didn't work, so the cause was rehashed, it being parallel tests sharing the process-wide ProcessorAffinity. [assembly: DoNotParallelize] fixed it.

Quality risks

- Exception handling in ProcessFrame catches all exceptions as skipped frames, which could hide real issues.
- The report-level NFR-08 check cant fail; per-condition behaviour is accounted for by dedicated tests.
- Simulator state (CPU affinity) is process-wide, which caused the concurrency bug
- Security surface is nonexistent. The real concern is AGPL licensing of the model and unreviewed third-party packages.

- Bias: the model is COCO-pretrained, and detection quality was tried on two images, so fairness across conditions weren't evaluated.

Reflection

AI really sped up the process of building the foundations and working with unfamiliar APIs as well as keeping track of what is written in the report itself rather than writing the report. The code output from Claude did set us back only slightly since it based its code on outdated documentation and older versions of the API, but that's where Copilot came in. Given its direct access to code in Visual Studio, as well as up-to-date documentation, the issue was quickly resolved. The temporary thresholds were not an issue considering there was no data available to back the placeholders. Once data was available, they were reconsidered. The concurrency bug was a concern because there would have been a chance that it could have slipped our attention and remained an issue, therefore, producing improper results. If we could do something differently, it would be to rely on GitHub Copilot for in-IDE development, and use Claude as a second opinion. Learning how to read library code by decompiling it, analysing tests written by AI, and AI output being inaccurate regardless of what you use helped bring this project to what it is.

Task 10: Quality Improvement and Future Recommendations

- Identify the main quality lessons learned from the project.
- Recommend improvements to the development and QA process.
- Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability.
- Explain how the project could evolve toward stronger continuous quality engineering practice.

Task 11: Team Contribution and Professional Reflection

- Provide a contribution statement for each group member.
- Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks.
- Include a brief individual reflection from each group member.

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: https://github.com/ExplosiveSalad/Quality-Assured-Vision.git

Task 12: Project Presentation and Demonstration

Each group must prepare a 15-slide presentation and demonstrate the completed project. The presentation should clearly summarise the problem, final solution, quality assurance activities, testing evidence, and final quality evaluation. The presentation should cover:

- Project title, group members, problem, stakeholders, and scope
- Key functional and non-functional requirements
- Final prototype, implemented features, and changes after Assessment 1
- Testing approach, coverage, execution results, and key findings
- Automation, CI/CD, or equivalent quality workflow
- Defects, root cause analysis, and corrective actions
- Quality metrics, advanced testing, and release decision
- GitHub Copilot use, limitations, risks, and future improvements
- Individual contributions and lessons learned.

All students must participate in the demonstration and show the working prototype or quality improvement solution. The demo should include key features, important test cases, automated tests or CI results where applicable, defect fixes, and evidence that major requirements have been satisfied. Students must be prepared to answer questions about the project, code, testing, quality artefacts, AI-assisted development, and individual contributions.

Deliverables

- Final Project and Software Quality Assurance Report: *6000--7000 words*
- Source code and GitHub repository link; provide evidence from GitHub commits.
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report